

ENTERPRISE PLATFORM

Low-Level Design Document

LLD — PART 2 OF 2

Sequence Diagrams | Error Catalogue | Data Flow | Test Strategy

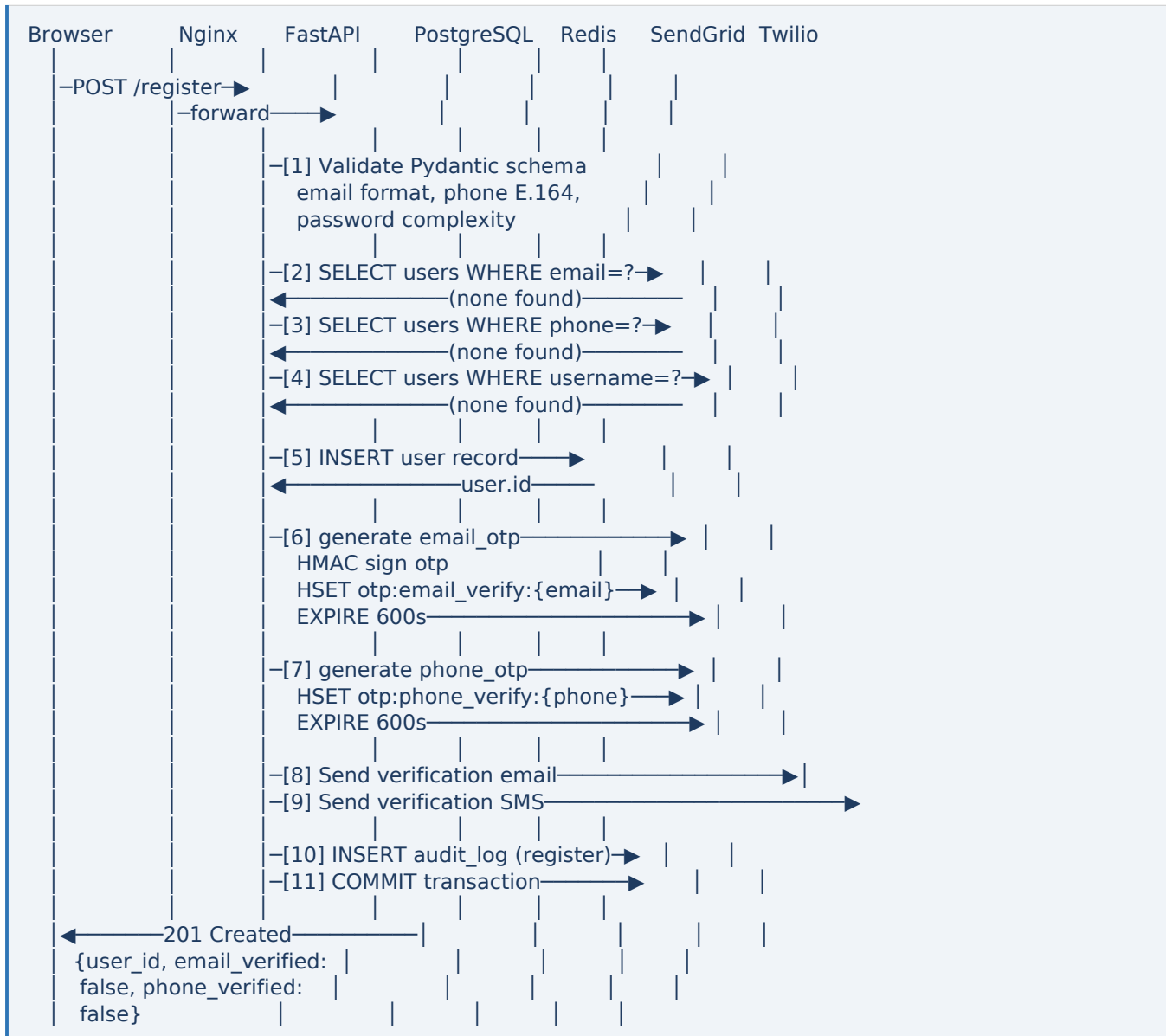
Document Scope

This is Part 2 of the 2-part LLD. Part 1 defined the database schema, API contracts, service interfaces, and data models. This part defines the step-by-step sequence of every major flow, the complete error handling catalogue, data flow diagrams, security implementation specifics, and the complete test strategy with test cases for every endpoint.

1. Sequence Diagrams — Every Major Flow

Each sequence diagram shows exactly what happens inside the system when a user performs an action. Every step is numbered. Every component involved is named. Every database and cache operation is shown.

1.1 User Registration Flow



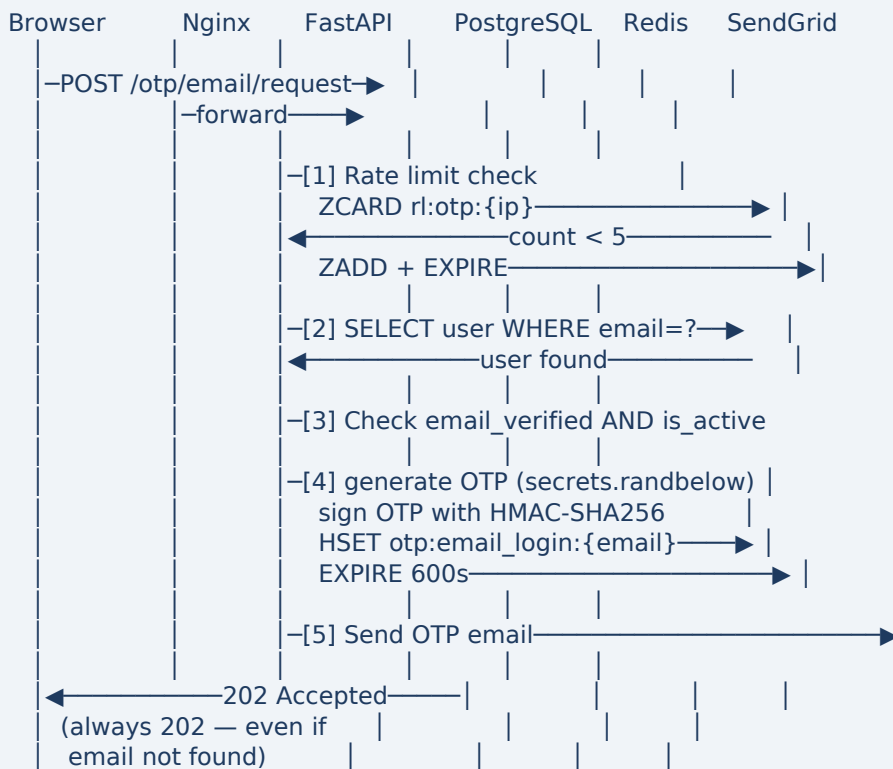
Failure paths in registration:

Step	Failure Condition	Response
------	-------------------	----------

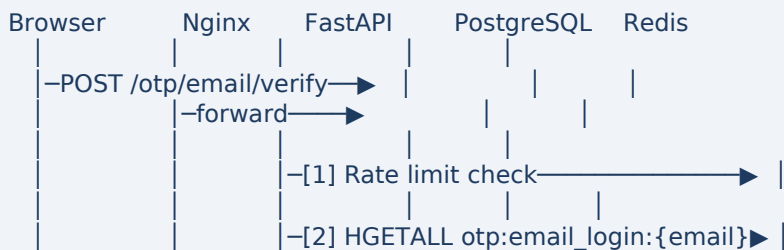
Step	Failure Condition	Response
Step 1	Pydantic validation fails	422 VALIDATION_ERROR — field and message returned
Step 2-4	Duplicate field found	409 DUPLICATE_FIELD — which field is duplicated
Step 5	Database insert fails	500 INTERNAL_ERROR — transaction rolled back
Step 8-9	Email or SMS fails	202 still returned — retry queue handles delivery

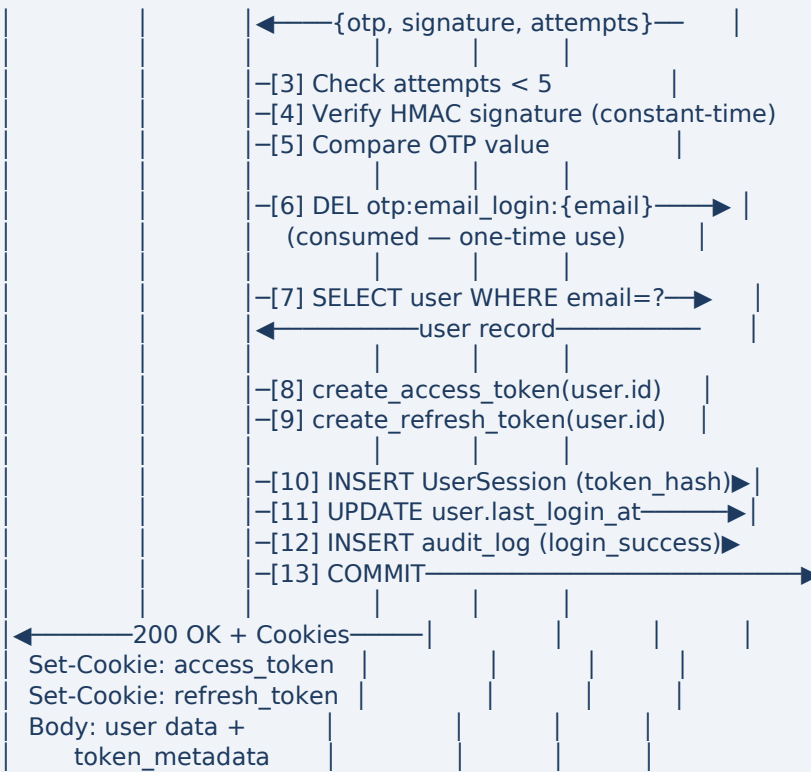
1.2 Email OTP Login Flow — Full Two-Step Sequence

== STEP 1: Request OTP ==

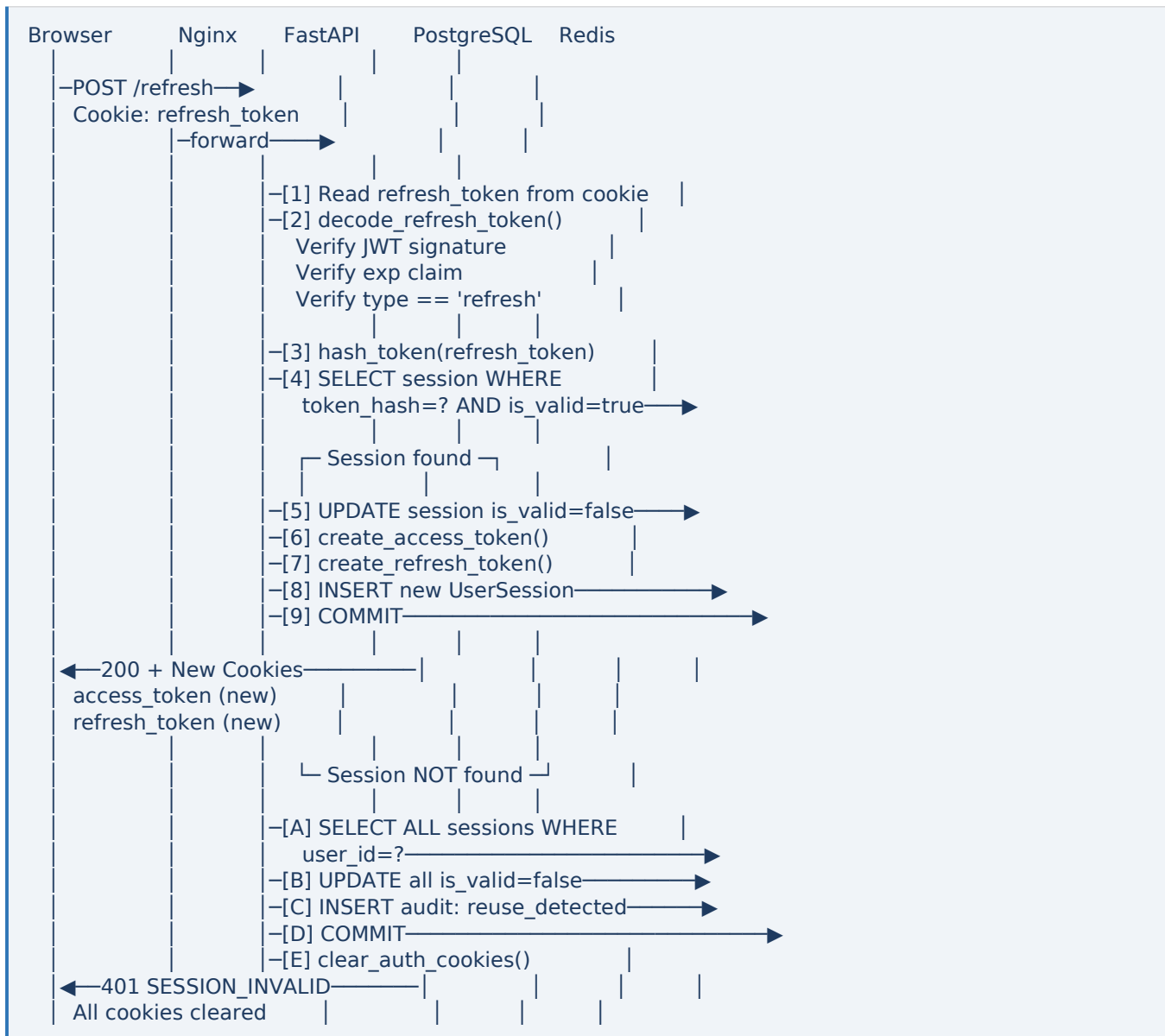


== STEP 2: Verify OTP ==



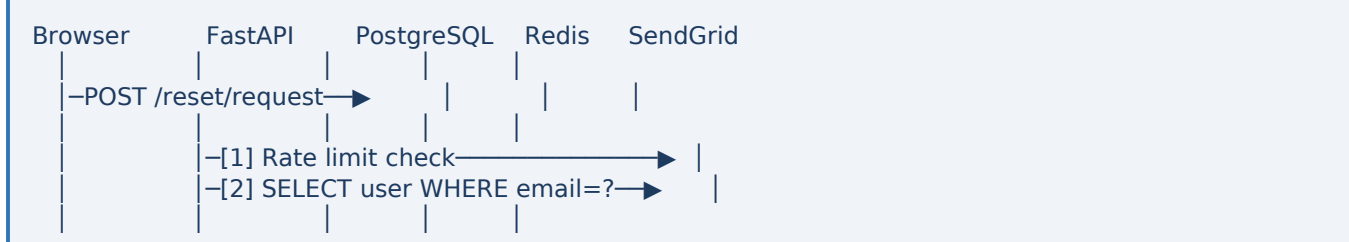


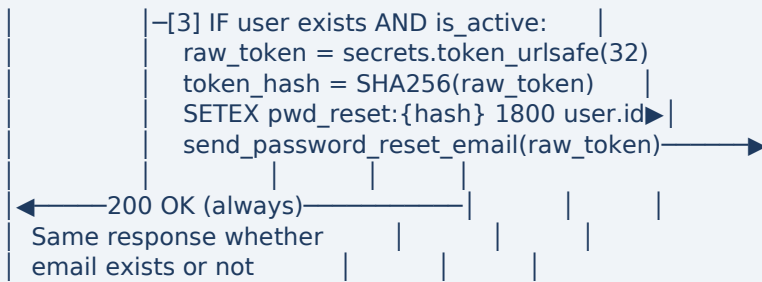
1.3 Token Refresh Flow



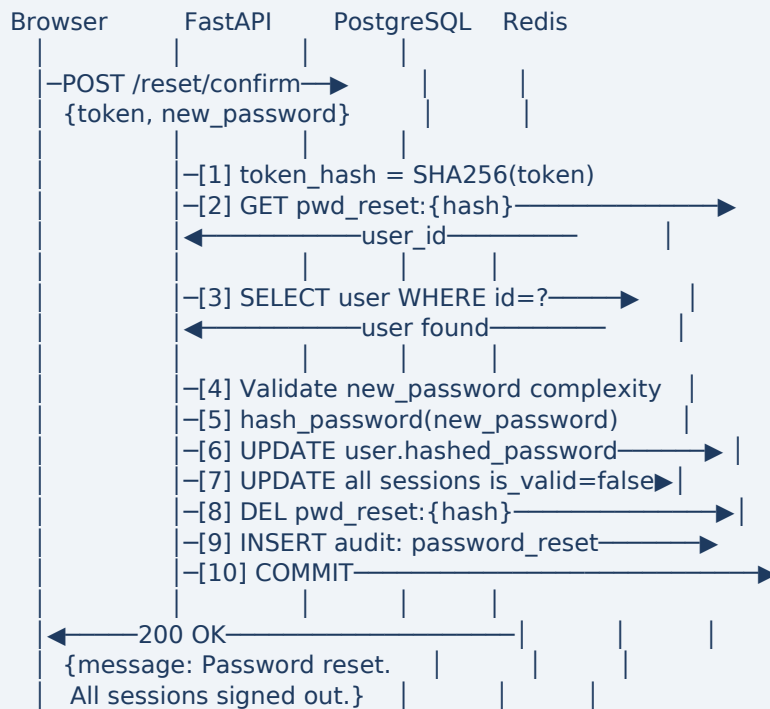
1.4 Password Reset Flow

== STEP 1: Request Reset ==

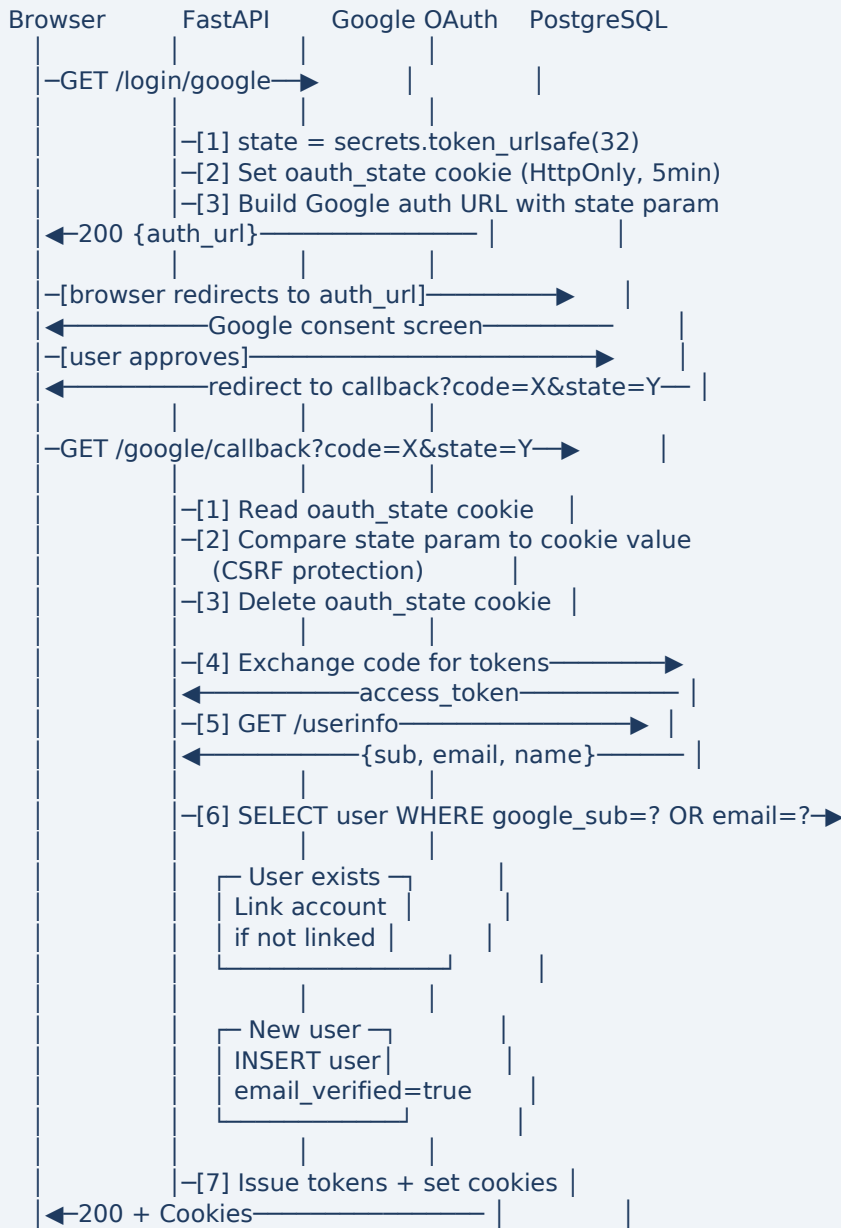




== STEP 2: Confirm Reset ==



1.5 Google OAuth Flow



2. Error Handling Catalogue

Every error the API can return is listed here with its HTTP status code, error code string, message template, and which endpoint produces it. Frontend and mobile developers use this catalogue to build error handling.

2.1 400 Bad Request Errors

Error Code	Message	Produced By
INVALID_TOKEN	Token is invalid or has expired.	verify/email, password-reset/confirm
INVALID_OTP	OTP is invalid, expired, or already used.	verify/phone, otp/email/verify, otp/sms/verify
MALFORMED_TOKEN	Malformed verification token.	verify/email
ACCOUNT_INACTIVE	This account has been deactivated.	password-reset/confirm
INVALID_STATE	OAuth state mismatch. Possible CSRF attack.	google/callback

2.2 401 Unauthorized Errors

Error Code	Message	Produced By
INVALID_CREDENTIALS	Invalid username or password.	login/password
INVALID_OTP	OTP is invalid, expired, or already used.	otp/*/verify
NOT_AUTHENTICATED	Not authenticated.	me, refresh, logout
INVALID_TOKEN	Invalid or expired token.	me, refresh
SESSION_INVALID	Session has been revoked.	refresh — reuse detected
NO_TOKEN	Not authenticated.	refresh — no cookie

2.3 403 Forbidden Errors

Error Code	Message	Produced By
ACCOUNT_NOT_VERIFIED	Please verify email and phone before logging in.	login/* — unverified account
ACCOUNT_INACTIVE	This account has been	login/* — inactive account

Error Code	Message	Produced By
	deactivated.	
PERMISSION_DENIED	Permission required: {permission}	Any permission-gated endpoint

2.4 404 Not Found Errors

Error Code	Message	Produced By
USER_NOT_FOUND	User not found.	verify/email, verify/phone, me

2.5 409 Conflict Errors

Error Code	Message	Produced By
DUPLICATE_FIELD	This {field} is already registered.	register — duplicate email/phone/username

2.6 422 Validation Errors

Validation errors return a list of errors, not a single error. Each item identifies the specific field:

```
HTTP 422 Unprocessable Entity
{
  "errors": [
    {
      "code": "VALIDATION_ERROR",
      "field": "password",
      "message": "Password must contain: one uppercase letter, one digit"
    },
    {
      "code": "VALIDATION_ERROR",
      "field": "phone",
      "message": "Phone must be in E.164 format, e.g. +919876543210"
    }
  ]
}
```

2.7 429 Rate Limit Errors

```
HTTP 429 Too Many Requests
{
```

```
"error": {  
  "code":      "RATE_LIMIT_EXCEEDED",  
  "message":   "Too many requests. Max 5 per 900 seconds.",  
  "retry_after_seconds": 900,  
  "reset_at":  1705412460  
}
```

Headers always returned on rate-limited endpoints:

```
X-RateLimit-Limit:    5  
X-RateLimit-Remaining: 0  
X-RateLimit-Reset:    1705412460  
X-RateLimit-Window:   900  
Retry-After:          900 (only on 429 responses)
```

2.8 500 Internal Server Error

HTTP 500 Internal Server Error

```
{  
  "error": {  
    "code":      "INTERNAL_ERROR",  
    "message":   "An unexpected error occurred.",  
    "request_id": "uuid" ← user quotes this to support team  
  }  
}
```

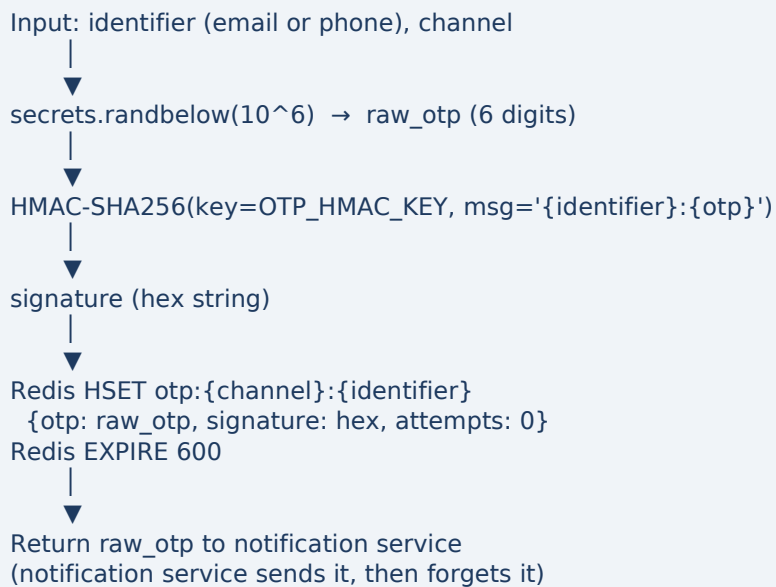
On every 500:

- Full traceback logged via structlog
- Error report emailed to support team
- Sentry captures the exception
- request_id links log entry to error report

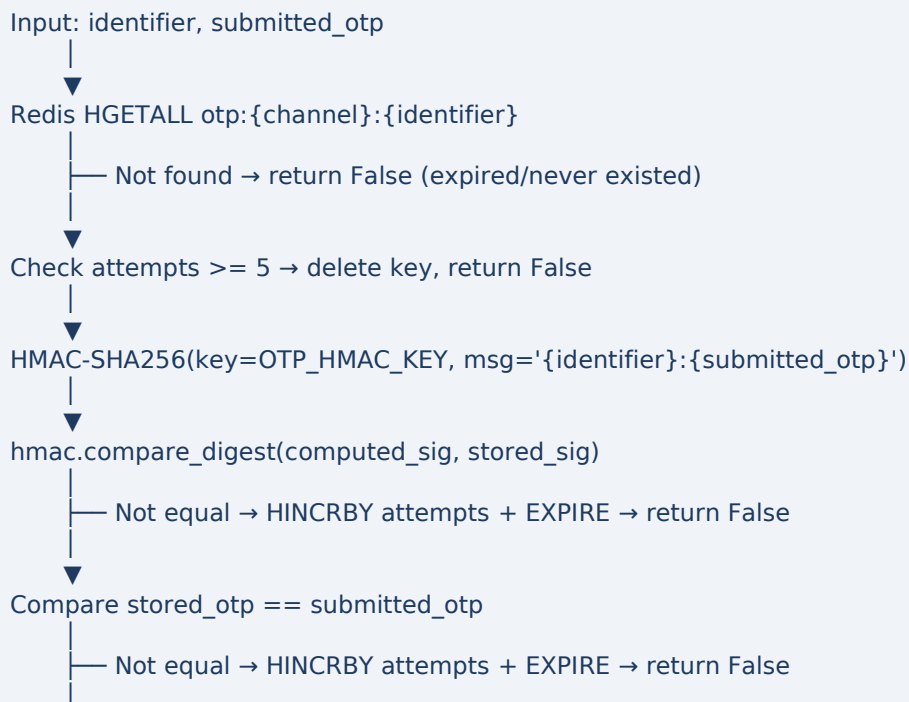
3. Data Flow Diagrams

3.1 OTP Generation and Verification Data Flow

OTP GENERATION:



OTP VERIFICATION:



```
graph TD
    A[DEL otp:{channel}:{identifier} ← consumed, one-time] --> B[return True]
```

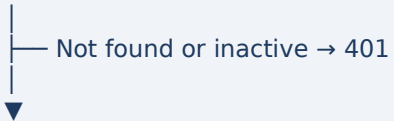
3.2 JWT Token Data Flow

TOKEN CREATION:

```
graph TD
    Input[Input: user_id] --> Payload["payload = {  
  sub: user_id,  
  type: 'access' | 'refresh',  
  iat: utcnow(),  
  exp: utcnow() + timedelta(minutes=15 | days=7),  
  jti: secrets.token_urlsafe(16) ← unique token ID  
}"]
    Payload --> Encode["jwt.encode(payload, SECRET_KEY, algorithm=HS256)"]
    Encode --> RawToken["raw_token (string)"]
    RawToken --> Access["access_token → Set-Cookie (HttpOnly, path=/)"]
    RawToken --> Refresh["refresh_token → hash_token(raw) → store in DB  
→ Set-Cookie (HttpOnly, path=/api/v1/auth/refresh)"]
```

TOKEN VALIDATION:

```
graph TD
    Input[Input: raw_token from cookie] --> Decode["jwt.decode(token, SECRET_KEY, algorithms=[HS256])"]
    Decode --> JWTErr["JWTErr → 401 INVALID_TOKEN"]
    JWTErr --> CheckType["Check payload.type == 'access'"]
    CheckType --> WrongType["Wrong type → 401 INVALID_TOKEN"]
    WrongType --> ExtractSub["Extract payload.sub (user_id)"]
    ExtractSub --> SQL["SELECT user WHERE id = user_id"]
```



Return user object to endpoint

4. Test Strategy

The test strategy defines what must be tested, how it is tested, and what the minimum acceptable coverage is before any code is deployed to production.

4.1 Test Pyramid

Layer	Count	Tool	What Is Tested
Unit tests	60%	pytest	Individual functions: hash_password, generate_otp, sign_otp, decode_access_token, rate limit logic
Integration tests	30%	pytest + httpx AsyncClient	Full endpoint flows with mock Redis and SQLite in-memory database
End-to-end tests	10%	pytest + real test DB	Critical paths against real PostgreSQL and Redis in CI environment

4.2 Minimum Coverage Requirements

Module	Min Coverage	Reason
app/core/security.py	100%	All cryptographic functions must be fully tested
app/services/otp_service.py	100%	OTP logic is security-critical
app/api/v1/endpoints/auth.py	90%	All auth flows and error paths
app/core/rate_limiter.py	90%	Rate limiting must work correctly
app/middleware/	85%	Error handler and request ID
Overall project	80%	CI gate — deployment blocked if below

4.3 Critical Test Cases

#	Test Case	Expected Result	Why Critical
1	Register with duplicate email	409 DUPLICATE_FIELD, field=email	Prevent duplicate accounts
2	Register with weak password	422 VALIDATION_ERROR	Enforce security policy
3	Login with wrong password	401 INVALID_CREDENTIAL	Correct error, no user enumeration

#	Test Case	Expected Result	Why Critical
		S	
4	Login with unknown email	401 INVALID_CREDENTIAL S (not 404)	Prevent user enumeration
5	Verify OTP — wrong code	400 INVALID_OTP	Reject wrong OTP
6	Verify OTP — correct code twice	401 on second attempt	One-time use enforcement
7	Verify OTP — 5 wrong attempts	Key deleted, 400 on 6th attempt	Brute force protection
8	Login before email verified	403 ACCOUNT_NOT_VERIFIED	Verification gate enforced
9	Access /me without cookie	401 NOT_AUTHENTICATED	Auth guard works
10	Access /me with expired token	401 INVALID_TOKEN	Token expiry enforced
11	Refresh — old token after rotation	401 SESSION_INVALID	Rotation prevents reuse
12	Refresh — reuse detection	401, all sessions killed	Reuse attack mitigation
13	OTP request — unknown email	202 (not 404)	Anti-enumeration
14	Rate limit — 6th OTP request	429 RATE_LIMIT_EXCEEDED	Rate limiting works
15	Rate limit headers present	X-RateLimit-Limit in response	Headers implemented
16	X-Request-ID in every response	UUID in X-Request-ID header	Tracing implemented
17	SQL injection in username field	422 or no data leak	Injection prevention
18	Password reset — wrong token	400 INVALID_TOKEN	Token validation works
19	Password reset — all sessions invalidated	Sessions is_valid=false	Session cleanup works
20	Google OAuth — state mismatch	400 INVALID_STATE	CSRF protection works

4.4 Running the Test Suite

```
# Install test dependencies
pip install aiosqlite pytest-asyncio httpx

# Run all tests
pytest -v

# Run with coverage report
pytest -v --cov=app --cov-report=term-missing

# Run only security-critical tests
pytest -v -k 'otp or token or rate_limit or injection'

# Run only a specific file
pytest tests/test_login.py -v

# CI gate — fails if coverage below 80%
pytest --cov=app --cov-fail-under=80

# Generate HTML coverage report
pytest --cov=app --cov-report=html
open htmlcov/index.html
```

5. Security Implementation Details

5.1 Cryptographic Specifications

Operation	Algorithm	Parameters
Password hashing	bcrypt	Cost factor 12 (~250ms). Via passlib. Auto-upgrades on login.
JWT signing	HS256	SECRET_KEY minimum 64 chars. jti claim for future deny-listing.
OTP generation	CSPRNG	secrets.randbelow(10^6). Zero-padded to 6 digits.
OTP signing	HMAC-SHA256	Key: OTP_HMAC_KEY minimum 32 chars. Constant-time verification.
Token hashing	SHA-256	hashlib.sha256. For refresh token and password reset token storage.
Reset token generation	CSPRNG	secrets.token_urlsafe(32) — 256 bits of entropy.
State token (OAuth)	CSPRNG	secrets.token_urlsafe(32). Validated with secrets.compare_digest.
Timing attack defence	constant-time comparison	hmac.compare_digest for all secret comparisons.

5.2 Cookie Security Specifications

Cookie	HttpOnly	Secure	SameSite
access_token	true	true (prod)	Strict
refresh_token	true	true (prod)	Strict
oauth_state	true	true (prod)	Lax (required for OAuth redirect)

access_token path: / (sent with every request to the API)

refresh_token path: /api/v1/auth/refresh (sent ONLY to the refresh endpoint)

6. Complete Document Set Summary

The HLD and LLD document set for Module 1 is now complete. This table provides the index for the entire documentation suite.

Doc	Title	Contents
HLD 1	Architecture and Topology	Executive summary, architecture decision, system topology, module map, deployment phases, API strategy, NFRs, compliance
HLD 2	Components and Data	Component design, data architecture, multi-tenancy, security architecture, event-driven messaging, caching, observability
HLD 3	Scale and Evolution	Scalability strategy, cost model, module integration contracts, evolution roadmap, team structure, risk register
LLD 1	Schema and API Contracts	Database schema, all API endpoint contracts, service interfaces, Pydantic validation rules, Redis data models
LLD 2	Flows and Tests (this doc)	Sequence diagrams, error catalogue, data flow, security specs, test strategy, test cases

Module 1 Documentation Complete

All five documents together form the complete design specification for Module 1. When starting Module 2 (User Profiles), refer to HLD Part 2 Section 18.1 for the integration contract and HLD Part 2 Section 13 for the event catalogue. The shared kernel interfaces defined in LLD Part 1 Section 3 are the entry points for all new modules.